

LIBRARY CLASSES – WRAPPER CLASSES

Wrapper Classes in Java

Wrapper classes are used to convert **primitive data types** into **objects** and vice versa. They are part of the java.lang package. Wrapper classes **encapsulate primitive data types** (like int, float, char, etc.) into **objects**.

Primitive Type Wrapper Class

byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Autoboxing and Unboxing

Autoboxing:

- **Definition:** Automatic conversion of a **primitive type** into its corresponding **wrapper class object**.
- **Example 1:**

```
int a = 10;
```

```
Integer a2 = new Integer(a); // older style, not recommended (Known as boxing)
```

- **Example 2:**

```
Integer a3 = 5; // autoboxing from int to Integer
```

Unboxing:

- **Definition:** Automatic conversion of a **wrapper class object** back into a **primitive type**.
- **Example:**

```
Integer i = new Integer(50); // Here i stores an Integer Object
```

```
int a = i; // unboxing from Integer to int
```

Useful Character Class Methods

These methods operate on char values and help with character classification and conversion:

Method	Description	Example
Character.toUpperCase(char)	Converts a character to uppercase	'a' → 'A'
Character.toLowerCase(char)	Converts a character to lowercase	'Z' → 'z'
Character.isUpperCase(char)	Checks if the character is uppercase	'A' → true, 'a' → false
Character.isLowerCase(char)	Checks if the character is lowercase	'z' → true
Character.isWhitespace(char)	Checks if the character is a whitespace	' ' → true
Character.isDigit(char)	Checks if the character is a digit	'5' → true
Character.isLetter(char)	Checks if the character is a letter	'a' → true, '1' → false
Character.isLetterOrDigit(char)	Checks if the character is a letter or digit	'a', '1' → true

Converting Strings to Primitive Types

Use the `parseDataType()` methods of wrapper classes to convert strings to primitive values.

Syntax:

```
data_type var = WrapperClass.parseDataType("String");
```

Examples:

```
int a = Integer.parseInt("10");
```

```
float f = Float.parseFloat("10.5");
```

```
double d = Double.parseDouble("15.06");
```

```
String s = "20";
```

```
int x = Integer.parseInt(s);
```

```
float y = Float.parseFloat(s);
```

Alternative: `valueOf()` Method

- Returns **wrapper class object** instead of a primitive type.
- Can also be used for conversion from String

Examples:

```
Integer a = Integer.valueOf("10");
```

```
Float f = Float.valueOf("10.5");
```

```
Double d = Double.valueOf("20.3");
```

Converting Primitives to Strings

You can convert numbers to String using:

1. toString() method:

```
String s1 = Integer.toString(10);
```

```
String s2 = Double.toString(13.12);
```

```
String s2= Float.toString(12.3);
```

```
String s4= Long.toString(50);
```

2. String.valueOf() method (more flexible):

```
int n = 123;
```

```
String s = String.valueOf(n); // works for all types (converts any primitive type to String)
```

```
double d = 45.67;
```

```
String str = String.valueOf(d);
```

```
long k=50L;
```

```
String s5= String.valueOf(k);
```

Difference between parseDataType() and valueOf()

Feature / Criteria	parseDataType()	valueOf()
Return Type	Returns a primitive type	Returns a wrapper class object
Return Example	int, float, double, etc.	Integer, Float, Double, etc.
Usage	To get a primitive from a String	To get a wrapper object from a String
Example	int x = Integer.parseInt("10");	Integer x = Integer.valueOf("10");

**Both valueOf() and parseDataType can be used to convert String to primitive (call with appropriate Wrapper class)

Possible Errors: (Runtime Errors)

Code Example	Method Used	Error Type	Explanation
Integer.parseInt("abc");	parseInt()	NumberFormatException	"abc" is not a number
Integer.parseInt("10.5");	parseInt()	NumberFormatException	Decimal value in integer parsing
Integer.parseInt("");	parseInt()	NumberFormatException	Empty string is invalid
Integer.parseInt(null);	parseInt()	NumberFormatException / NullPointerException	Null input
Float.parseFloat("12.3abc");	parseFloat()	NumberFormatException	Trailing non-numeric characters

Java Library Classes – A Brief Overview

What are Library Classes?

- **Library classes** are **predefined classes** in Java that are part of the **Java API (Application Programming Interface)**.
- These classes provide **ready-made functionality** to handle common programming tasks like:
 - Math operations
 - String manipulation
 - Data structures
 - File handling
 - Input/Output
 - Networking
 - GUI programming

These classes are grouped into **packages** (e.g., java.lang, java.util, java.io).

Commonly Used Java Library Classes

Class Name	Package	Purpose / Use Case
String	java.lang	For creating and manipulating text
Math	java.lang	Provides math functions like sqrt(), pow(), etc.

Integer, Double, Character, etc.	java.lang	Wrapper classes for primitive types
ArrayList, HashMap, HashSet	java.util	Data structures for storing and managing collections
Scanner	java.util	For taking input from the user
Random	java.util	For generating random numbers
File, FileReader, BufferedReader	java.io	File handling (read/write files)
LocalDate, LocalTime	java.time	Date and time manipulation

java.lang is automatically imported.

Other classes must be imported using import statement.

Useful for efficient, cleaner, and faster development.

Subhasis